# 4.i Classes – Special Classes – Enums

Kotlin Enum Classes

# Contents


1 **Enum Classes**
Type-safe enums with the enum class keyword

2 **Initialization**
Enum constants with constructor properties

3 **Anonymous Classes**
Overriding methods per enum constant

4 **Implementing Interfaces**
Common and per-entry interface implementations

5 **Working with Enum Constants**
entries, valueOf, name, ordinal, and generic access

# Enum Classes

The most basic use case for enum classes is the implementation of **type-safe enums**. Each enum constant is an **object**. Constants are separated by commas.

## Basic Declaration

```
enum class Direction {
    NORTH, SOUTH, WEST, EAST
}
```

## Using Enums in when Expressions

```
enum class Direction { NORTH, SOUTH, WEST, EAST }

fun describe(dir: Direction) = when (dir) {
    Direction.NORTH -> "Up"
    Direction.SOUTH -> "Down"
    Direction.WEST  -> "Left"
    Direction.EAST  -> "Right"
}
```

## Key Points

- Declared with **enum class** keyword
- Each constant is a **singleton object**
- Constants separated by commas

# Initialization

Since each enum is an **instance of the enum class**, it can be initialized with **constructor properties**.

## Enum with Properties

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF)
}
```

### Key Points

- Constructor defines **properties** for each constant
- Each constant passes its values to the constructor
- Properties are accessible via **dot notation**
- Enums can also define **methods**

## Enum with Members (semicolon required)

```
enum class Planet(val mass: Double, val radius: Double) {
    EARTH(5.976e24, 6.37814e6),
    MARS(6.421e23,  3.3972e6);  // semicolon separates constants from members

    fun surfaceGravity(): Double = 6.67300E-11 * mass / (radius * radius)
}
```

# Anonymous Classes

Enum constants can declare their own **anonymous classes** with their corresponding methods, as well as **overriding** base methods.

```
enum class ProtocolState {
    WAITING {
        override fun signal() = TALKING
    },

    TALKING {
        override fun signal() = WAITING
    };

    abstract fun signal(): ProtocolState
}
```

## Key Points

- Each constant opens a **{ }** block to override methods
- **Semicolon required** to separate constants from member definitions
- The enum class can declare **abstract** methods
- Each constant **must** override all abstract members
- Different **behavior per constant** — like a state machine

# Implementing Interfaces

An enum class can **implement an interface** (but **cannot derive from a class**), providing either a common implementation for all entries, or separate implementations per entry.

```
enum class IntArithmetics
    : BinaryOperator<Int>, IntBinaryOperator {

    PLUS {
        override fun apply(t: Int, u: Int): Int = t + u
    },
    TIMES {
        override fun apply(t: Int, u: Int): Int = t * u
    };

    override fun applyAsInt(t: Int, u: Int) = apply(t, u)
}
```

## Key Points

- Can implement **one or more interfaces**
- **Cannot extend** a class (implicitly extends Enum)
- Common implementation in enum body
- Or **per-entry** implementation in anonymous class
- All enums implement **Comparable** by default (natural order)

# Working with Enum Constants

Enum classes have **synthetic properties and methods** for listing the defined enum constants and getting an enum constant by its name.

## Signatures

```
EnumClass.valueOf(value: String): EnumClass
EnumClass.entries: EnumEntries<EnumClass>   // specialized List
```

## Usage Example

```
enum class RGB { RED, GREEN, BLUE }

fun main() {
    // Iterate all entries
    for (color in RGB.entries) println(color)  // RED, GREEN, BLUE

    // Get by name (throws IllegalArgumentException if not found)
    println(RGB.valueOf("RED"))    // RED
}
```

# Working with Enum Constants (cont.)

## name & ordinal Properties

```
enum class RGB { RED, GREEN, BLUE }

println(RGB.RED.name)      // RED
println(RGB.RED.ordinal)   // 0
```

- **name** — the declared name of the constant
- **ordinal** — position (starting from 0)
- Available on **every** enum constant

## Generic Access with enumEntries<T>()

```
enum class RGB { RED, GREEN, BLUE }

// Generic access (Kotlin 2.0+)
inline fun <reified T : Enum<T>> printAllValues() {
    println(enumEntries<T>().joinToString { it.name })
}

printAllValues<RGB>()  // RED, GREEN, BLUE
```

⚠️ **enumEntries<T>()** replaces **enumValues<T>()** — returns the same cached list each call instead of creating a new array.

# Practice Exercises

# Exercise 1: Enum with when

Define a Season enum (SPRING, SUMMER, AUTUMN, WINTER) and write a function that returns a description for each season using a when expression.

## Solution

```
enum class Season {
    SPRING, SUMMER, AUTUMN, WINTER
}

fun describe(s: Season): String = when (s) {
    Season.SPRING -> "Flowers bloom"
    Season.SUMMER -> "Sun shines"
    Season.AUTUMN -> "Leaves fall"
    Season.WINTER -> "Snow falls"
}

fun main() {
    println(describe(Season.WINTER))
}
```

### Concepts Used:

- Basic enum class declaration
- when expression with enum
- Exhaustive matching (no else needed)
- Type-safe constant references

**Output:** `Snow falls`

# Exercise 2: Enum with Properties

Create an HttpStatus enum with a code (Int) and message (String). Add a method isSuccess() that returns true for codes in 200..299. Iterate all entries and print each.

## Solution

```kotlin
enum class HttpStatus(val code: Int, val message: String) {
    OK(200, "OK"),
    CREATED(201, "Created"),
    NOT_FOUND(404, "Not Found"),
    SERVER_ERROR(500, "Internal Server Error");

    fun isSuccess(): Boolean = code in 200..299
}

fun main() {
    for (status in HttpStatus.entries) {
        val ok = if (status.isSuccess()) "✓" else "✗"
        println("$ok ${status.code} ${status.message}")
    }
}
```

### Concepts Used:

- Enum with constructor properties
- Semicolon to separate members
- Custom method on enum
- entries iteration (Kotlin 1.9+)
- Range check with in operator

**Output:**

```
✓ 200 OK
✓ 201 Created
✗ 404 Not Found
✗ 500 Internal Server Error
```

# Exercise 3: Enum Implementing Interface

Define a Printable interface with a prettyPrint() method. Create a Color enum (RED, GREEN, BLUE) with a hex property that implements Printable, printing each color with its hex code.

## Solution

```
interface Printable {
    fun prettyPrint(): String
}

enum class Color(val hex: String) : Printable {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF");

    override fun prettyPrint(): String =
        "$name ($hex)"
}

fun main() {
    Color.entries.forEach {
        println(it.prettyPrint())
    }
}
```

### Concepts Used:

- Interface implementation on enum
- override keyword
- String templates with properties
- entries + forEach iteration
- Constructor properties on enum

**Output:**

```
RED (#FF0000)
GREEN (#00FF00)
BLUE (#0000FF)
```

# Your Turn — Practice Exercises

## 1 Enum State Machine

Create a TrafficLight enum (RED, YELLOW, GREEN). Add a next() method that cycles to the next light. Print five transitions starting from RED.

*Hint: Use when to return the next state.*

## 2 Enum with Abstract Method

Create a Shape enum (CIRCLE, SQUARE, TRIANGLE) where each constant overrides an abstract method area(size: Double) with the correct formula.

*Hint: Use anonymous class syntax for each constant.*

## 3 Enum valueOf Error Handling

Write a safe function that converts a user string to a Direction enum (NORTH, SOUTH, EAST, WEST) and handles invalid input gracefully.

*Hint: valueOf() throws IllegalArgumentException.*

## 4 Enum Filtering with entries

Create a Priority enum (LOW, MEDIUM, HIGH, CRITICAL) with an urgency level (Int). Filter and print only priorities with urgency above a threshold.

*Hint: Use entries.filter { } for selection.*

# Challenge Exercises

## 5 Sealed vs Enum Design

You need to model payment methods: CreditCard, BankTransfer, and Crypto. Each has different data fields. Should you use an enum or sealed class? Implement both and compare.

*Hint: Enums have fixed instances; sealed classes allow per-instance data.*

## 6 Enum-Powered Calculator

Create an Operation enum (ADD, SUB, MUL, DIV) that implements a Computable interface with apply(a: Int, b: Int): Int. Each constant overrides apply() with the matching arithmetic. Build a REPL that reads operator + operands and computes the result.

*Hint: Combine interface implementation, anonymous classes, and valueOf().*

## Tips for Success

- Experiment in the Kotlin Playground
- Try adding properties and methods to enums
- Read the official docs for edge cases
- Compare enum classes with sealed classes
- Practice valueOf() error handling patterns
- Test exhaustive when with and without else